SCS3305
Lecture I: Overview of Compilation

1

What is a Compiler?

- A compiler is a program that gets as input a program written in one language (the source language), and translates it into an equivalent program in another language (the target language).
- Usually, the source language is a high-level language program and the target language is a low level language e.g. Assembly language or machine language.
- A compiler is used to implement a high-level language on a computer.
- High-level languages are more suitable for humans to work with
- Computers execute instructions in machine language
- As an important part of the translation process, the compiler reports to its user the presence of any errors in the source program.

2

Reasons for using high-level languages

1. Compared to machine code, the notation used by programming languages is closer to the way humans think about problems.
2. Programs written in a high-level language tend to be shorter than equivalent programs written in machine code.
3. Another advantage of using a high-level level language is that the same program can be compiled to many different machine languages (native code) and, hence run on many different machines.
4. Compilers can be used to optimize code.

3

Reasons for Studying Compiler Construction

1. A good craftsman/craftswoman should know his/her tools, and compilers are important tools for programmers and computer scientists.
2. The techniques used for constructing a compiler are useful for other purposes as well e.g. natural language processing
3. There is a good chance that a programmer or computer scientist will need to write a compiler or interpreter for a domain-specific language.

4

The Analysis Synthesis Model of Compilation

- Compilation usually has 2 parts: Analysis and Synthesis
- Analysis breaks the HLL program up into constituent pieces and creates an intermediate representation
  - The ***front-end*** part of the compiler performs the *analysis* while the ***back-end*** part performs the *synthesis*
  - The analysis creates an *intermediate representation* of the source code which becomes the input to the back-end
- Synthesis constructs the desired target program from the intermediate representation

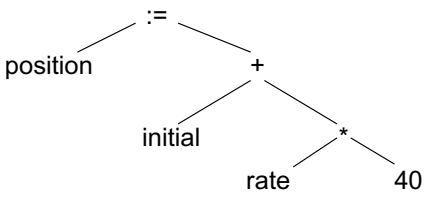
5

The Analysis Synthesis Model of Compilation

- During Analysis, the operations implied by the source program are determined and recorded in a hierarchical structure called a syntax tree (an intermediate representation)
- In a syntax tree, each node represents an operation, and the children of the node represent the arguments of the operation

6

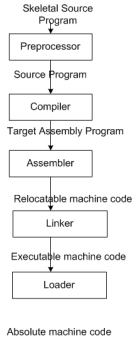
The Analysis Synthesis Model of Compilation



Example syntax Tree for position:=initial+rate*40

7

The context of a compiler



8

The context of a compiler

- A source program may consist of modules stored in separate source files
- The task of putting together the source program files is sometimes entrusted to a preprocessor
  - It may expand short hand notations, called macros into source language statements
- The target program created by the compiler may require further processing, e.g. if the target language is assembly code.
- An assembler may thus be required to create relocatable machine code (***object code***)
- The *relocatable machine code* may need to be combined with other machine code components e.g. libraries to create the executable code, and this is done by a linker.
- A loader loads the executable code in memory for execution to start

9

The context of a compiler

Some Compilation Tools …

- **Loader:** It is part of the operating system and is responsible for loading executable files into memory and starting the execution. Calculates the size of the programme and creates space for it in memory. It uses appropriate registers and then initiates the execution.
- **Source to Source Compiler**: Takes a source programme in one language and converts it to a source programme in another high level language

10

The context of a compiler

Some Compilation Tools

- **Pre-processor**: Deals with issues such as macro expansion and file inclusion.
  - Usually considered part of the compiler
- **Interpreter:** Differs from the compiler in that it reads the source code one line/statement at a time and converts it to machine code. This line of code is then executed. The interpreter then goes back and reads another line of code and repeats the process
- **Assembler:** Translates assembly language program to machine code. The output of an assembler is called an object file.
- **Linker:** merges various object files together to create an executable file. Determines the referenced modules in object files and the locations where they will be loaded hence ensuring instructions have absolute addresses.

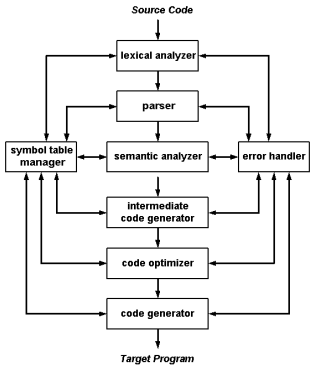
11

Grouping of Compilation Phases

- Compilation can be broken into two stages:
  - Analysis (Front End)
    - Lexical Analysis
    - Syntax Analysis
    - Semantic Analysis
    - Intermediate Code Generation
  - Synthesis (Back End)
    - Code Optimization
    - Code Generation

12

Grouping of Compilation Phases



13

Compilation Front-End

- The analyzer (front-end)
  - Recognises legal constructs
  - Reports errors
  - Produces Intermediate Language
  - Generates a preliminary storage map
- Intermediate code is simple machine-independent intermediate language.

14

Analysis of the source program

- In compilation, analysis has 3 stages: Lexical analysis, Syntax analysis and semantic analysis
- **Lexical Analysis (scanning)**
  - The stream of characters making the source program is read from left to right, and grouped into tokens, that are sequences of characters having a collective meaning.
  - E.g. for the statement *position:= initial + rate * 60* the following output is generated
  - *<id, position>, <:=>, <id, initial>, <+>, <id, rate>, <*> <num, 60>*
  - It can be said that the sequence of characters correspond to a word, while the token corresponds to a part of speech

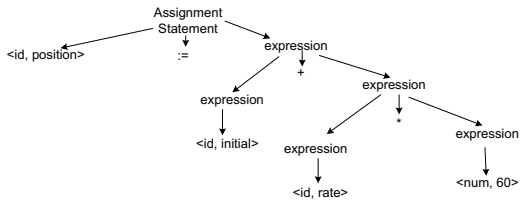
15

Analysis of the source program

- **Syntax Analysis (parsing)**
- Performs a hierarchical analysis of the program.
- Involves grouping the tokens of the source program into grammatical phrases.
- Usually these groups of grammatical phrases are represented by a parse tree

16

Analysis of the source program



Parse tree: Hierarchical representation of the source program

17

Analysis of the source program

- **Syntax Analysis (parsing) …**
- The hierarchical structure of a program is usually expressed by recursive rules. E.g. the following rules give the definition of expressions:

1. *An id is an expression*
2. *A num is an expression*
3. *If expression1 and expression2 are expressions, then so are*

   expression1 + expression2

   *expression1 * expression2*

   (expression1)

18

Analysis of the source program

- **Syntax Analysis (parsing) …**
- The following rules can be used to define statements

1. *If identifier1 is an identifier, and expression2 is an expression then*

   Identifier1:=expression2 is a statement
2. *If expression1 is an expression and statement2 is a statement, then*

   ***while**(expression1) **do** statement2*

   ***if** (expression1) **then** statement2*

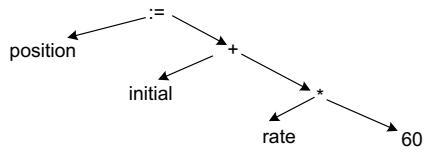
   are statements

- Syntax analysis is usually recursive, while lexical analysis is not

19

Analysis of the source program

- **Syntax Analysis (parsing) …**
- A more common representation of the syntactic structure is given by the syntax tree
- It is a compressed representation of the parse tree
  - The operators appear as interior nodes, and the operands of the operator are children nodes of the node for that operator



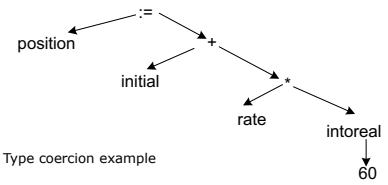
Syntax Tree

20

Analysis of the source program

Semantic Analysis

- ✓ Checks the program for semantic errors
- ✓ Gathers type information for the subsequent code-generation phase
- E.g. type checking to ensure that operators have permitted types
- Some *operand coercion* may be permitted in some languages e.g. conversion of an integer into a real



Type coercion example

21

Grouping of Compilation Phases

- Compilation can be broken into two stages:
  - Analysis (Front End)
    - Lexical Analysis
    - Syntax Analysis
    - Semantic Analysis
    - Intermediate Code Generation
  - **Synthesis (Back End)**
    - **Code Optimization**
    - **Code Generation**

22

Compilation Back-End

- The synthesizer (back-end)
  - ✓ Translates the intermediate language (IL) into target machine code
  - ✓ Chooses the instructions required for each IL operation
  - ✓ Decides what information to keep on the processor registers
  - ✓ Ensures that the resulting program uses the target system efficiently

23

Symbol Table Management

- A compiler needs to keep a record of the identifiers used in the source program, and collect information about the various attributes of each identifier
- These attributes may provide information about
  - The storage allocated for an identifier
  - Its type
  - Its scope
  - Access level
  - For procedure names
    - Number and types of arguments
    - The method of parsing each argument (reference, value)
    - Returned type

24

Symbol Table Management

- A symbol table is a data structure containing a record of each identifier, with fields for the attributes of the identifier
- It allows us to find the record for each identifier quickly, and to store and retrieve data on that record quickly
- When an identifier in the source program is recognized by the scanner, it is entered into the symbol table.
- The other attributes cannot normally be determined in the scanning phase.
- The remaining phases enter information about identifiers in the symbol table and use this information in various ways

25

Error Detection and Reporting

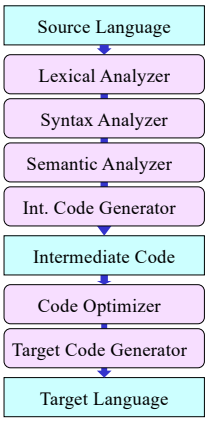
- Each phase encounters errors
- After detecting an error, a phase must somehow deal with the error, so that compilation can proceed allowing further errors in the program to be detected
- The lexical phase can detect errors where the characters remaining in the input do not form any token of the language
- Errors where the token stream violates the structure (syntax) rules of the language are handled by the parser

26

Error Detection and Reporting

- In semantic analysis, the compiler tries to detect constructs that have the right syntactic structure but no meaning to the operation involved e.g. adding two identifiers where one is a number and the other is an array name

27



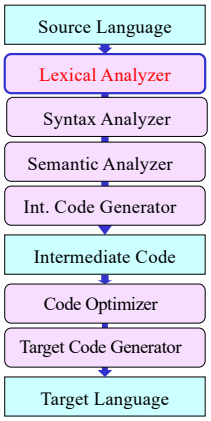
Compilation Example

- As the translation progresses, the internal representation of the source program changes.
- The following example illustrates the compilation process

Source Code:

```
cur_time = start_time + cycles * 60
```

28



Source Code:

```
cur_time = start_time + cycles * 60
```

Lexical Analysis:

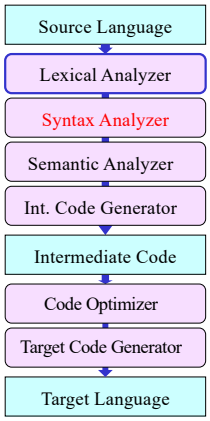
```
ID(1) ASSIGN ID(2) ADD ID(3) MULT
INT(60)
```

Symbol Table

1	cur_time	...
2	start_time	...
3	cycles	...

The 1 in id(1) means that the lexeme for the id is in position 1 of the symbol table

29



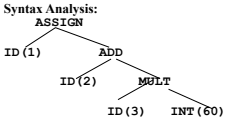
Source Code:

```
cur_time = start_time + cycles * 60
```

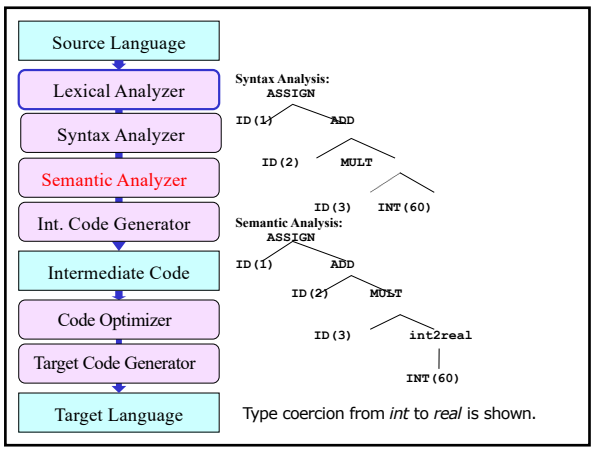
Lexical Analysis:

```
ID(1) ASSIGN ID(2) ADD ID(3) MULT INT(60)
```

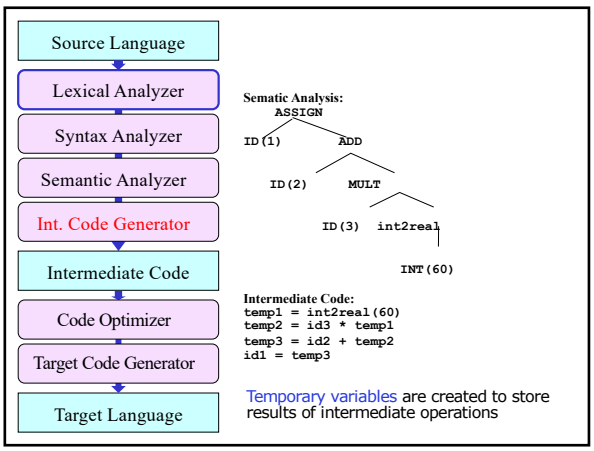
Syntax Analysis:



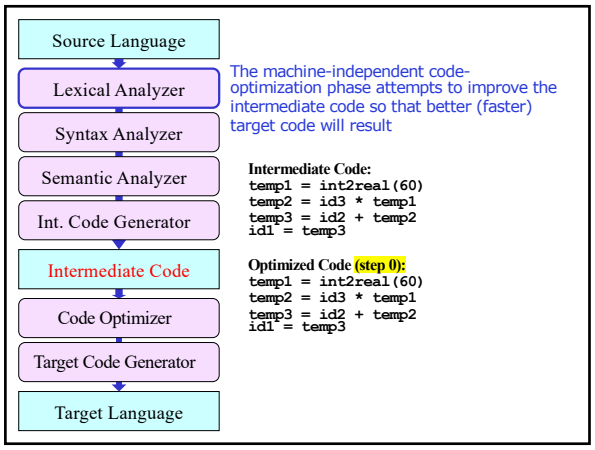
30



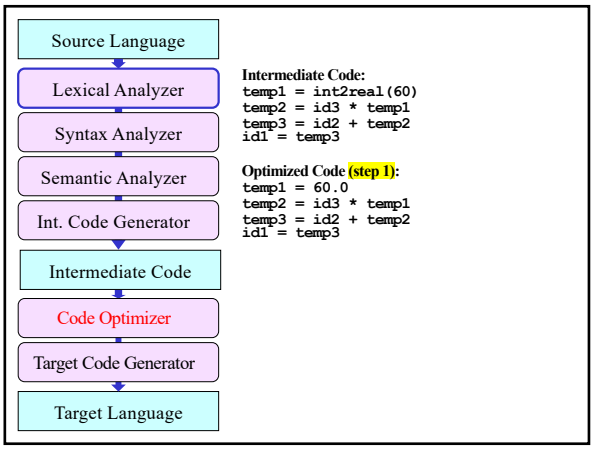
31



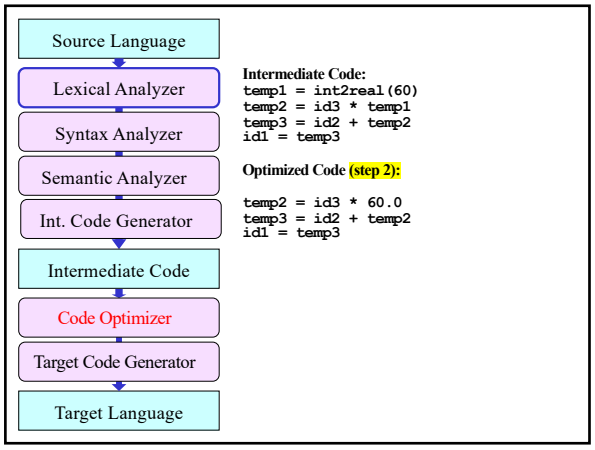
32



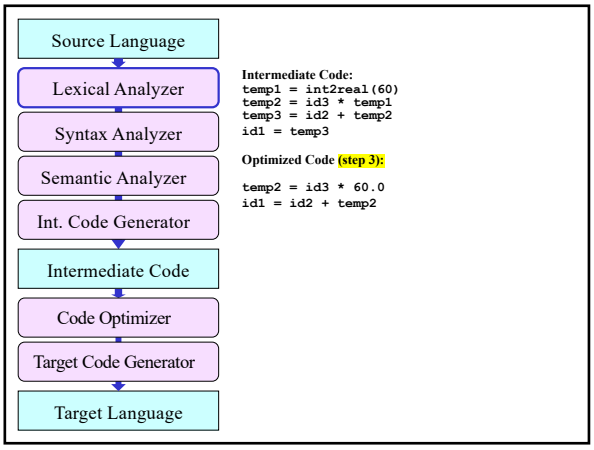
33



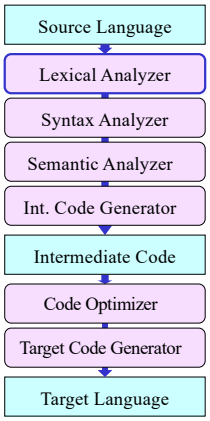
34



35



36



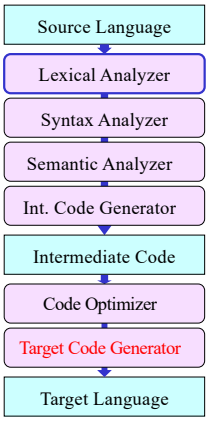
Intermediate Code:

```
temp1 = int2real(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

Optimized Code:

```
temp1 = id3 * 60.0
id1 = id2 + temp1
```

37



Intermediate Code:

```
temp1 = int2real(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

Optimized Code:

```
temp1 = id3 * 60.0
id1 = id2 + temp1
```

Target Code:

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

38

Passes

- Several phases of compilation are usually implemented in a single pass consisting of reading an input file and writing an output file
- The activities of compilation are usually interleaved during the pass
- For instance lexical analysis, syntax analysis, semantic analysis and intermediate code generation might be grouped into one pass
- We might think of the parser as being in charge.
  - It attempts to discover grammatical structure in the tokens it sees, it obtains the tokens as it needs them by calling the lexical analyzer to get the next token
  - As the grammatical structure is discovered, the parser calls the intermediate code generator to perform semantic analysis and generate a portion of the code

39

Passes

- Single pass compilers may require that the entire intermediate program and the intermediate internal form (parse trees) be kept in memory, this might not be feasible if the program and the intermediate forms occupy too much memory
- In some cases it might not be possible to produce code until we see the following sections of the code e.g. when a ***goto*** statement is used. Techniques such as backtracking (discussed later) can be used to overcome this problem

40

PASCAL Example

- Grammar (specified in BNF)
  - BNF grammar contains a set of rules that define the syntax of some construct in the programming language.

    ::= => is defined to be

    < > => non terminal symbols (constructs defined in the grammar)

    No brackets terminal symbols
- Sample program
- Lexical analysis
- Syntax analysis

41

SIMPLIFIED PASCAL GRAMMAR

- <program> ::= PROGRAM <prog-name> VAR <dec-list> BEGIN <stmt-list> END.
- <prog-name> ::= **id**
- <dec-list> ::= <dec> | <dec-list>; <dec>
- <dec> ::= <id-list> : <type>
- <type> ::= INTEGER
- <id-list> ::= **id** | <id-list> , **id**
- <stmt-list> ::= <stmt> | <stmt-list> ; <stmt>
- <stmt> ::= <assign> | <read> | <write> | <for>
- <assign> ::= **id** := <exp>
- <exp> ::= <term> | <exp> + <term> | <exp> - <term>
- <term> ::= <factor> | <term> * <factor> | <term> DIV <factor>
- <factor> ::= **id** | **int** | (<exp>)
- <read> ::= READ (<id-list>)
- <write> ::= WRITE (<id-list>)
- <for> ::= FOR <index-exp> DO <body>
- <index-exp> ::= **id** := <exp> TO <exp>
- <body> ::= <stmt> | BEGIN <stmt-list> END

42

Example of a PASCAL program

```
PROGRAM STATS
VAR
SUM, SUMSQ, I,  VALUE, MEAN, VARIANCE: INTEGER
BEGIN
SUM:=0;
 SUMSQ:=0;
FOR I:=1 TO 100 DO
    BEGIN
        READ (VALUE);
        SUM:=SUM+VALUE;
        SUMSQ:=SUMSQ+VALUE*VALUE
    END;
MEAN:=SUM DIV 100;
VARIANCE:=SUMSQ DIV 100 – MEAN*MEAN;
WRITE (MEAN, VARIANCE)
END.
```

43

Exercise

- Draw the parse tree for the example Pascal program, using the specified BNF Pascal grammar.

44

Read up on…

- A brief overview of all phases of compilation.
- An overview of main challenges in compiler design.
- Regular Languages (Regex, FA)
- DFA, NFA and NFA => DFA conversion.

45